

Разработка конкурентных структур данных с блокировками

В этой главе

- Что означает разработка структур данных, допускающих конкурентные обращения.
- Рекомендации по разработке таких структур.
- Примеры реализации структур данных, допускающих конкурентные обращения.

В предыдущей главе были рассмотрены низкоуровневые особенности атомарных операций и модель памяти. В этой главе мы отдохнем от низкоуровневых подробностей (хотя они нам еще понадобятся в главе 7) и поразмышляем над структурами данных.

Выбор структуры данных для решения проблем программирования может стать ключевой составляющей всего решения, и проблемы параллельного программирования не исключение. Если предполагается обращаться к структуре данных сразу из нескольких потоков, то либо она должна быть полностью неизменяемой, чтобы данные никогда не модифицировались и никакой синхронизации не требовалось, либо программу следует разрабатывать так, чтобы гарантировать корректную синхронизацию изменений между потоками. Один из вариантов заключается в применении для защиты данных отдельного мьютекса и внешней блокировки с использованием приемов, рассмотренных в главах 3 и 4, а другой — в разработке самой структуры данных, допускающей конкурентные обращения.

При разработке структуры данных, допускающей конкурентные обращения, можно воспользоваться основными строительными блоками многопоточных приложений из предыдущих глав, например мьютексами и условными переменными. Разумеется, нам уже встречалось несколько примеров, показывающих, как создать комбинацию из этих строительных блоков для разработки структур данных, допускающих безопасный конкурентный доступ сразу из нескольких потоков.

Сначала в этой главе мы рассмотрим ряд общих рекомендаций по созданию конкурентных структур данных. Затем, прежде чем переходить к более сложным структурам данных, еще раз поговорим о разработке базовых структур данных в плане применения основных строительных блоков блокировок и условных переменных. В главе 7 будет показано, как для создания структур данных без блокировок можно вернуться к основам и воспользоваться атомарными операциями, рассмотренными в главе 5.

Теперь завершим вводную часть и перейдем к рассмотрению всех составляющих разработки структур данных, допускающих конкурентные обращения.

6.1. Что означает разработка структур данных для конкурентного доступа

В наипростейшем представлении разработка структуры данных, допускающих конкурентные обращения, ведется для предоставления доступа к ней со стороны сразу нескольких потоков, выполняющих одинаковые или разные операции. При этом гарантируется, что каждый поток видит непротиворечивое представление данной структуры. Потеря или повреждение данных не допускаются, поддерживаются все инварианты и исключается возникновение проблемного состояния гонки за доступ к данным. Такая структура данных называется *потокобезопасной*. В общем, структура данных будет безопасна только для определенных видов конкурентного доступа. Вполне возможно, что несколько потоков будут одновременно выполнять над структурой данных один тип операции, а другая операция потребует исключительного доступа к ней одного из потоков. В иных случаях конкурентный доступ к структуре данных сразу нескольких потоков, выполняющих *разные* действия, может быть вполне безопасен, а обращение нескольких потоков, выполняющих *одно и то же* действие, вызовет проблемы.

Но это еще не все особенности разработки с учетом конкурентности. Здесь также уделяется внимание предоставлению потокам, обращающимся к структуре данных, возможности *конкурентных обращений*. В соответствии со своей природой мьютекс обеспечивает возможность *взаимного исключения*: в определенный момент завладеть блокировкой мьютекса может только один поток. Мьютекс защищает структуру данных, явно *предотвращая* конкурентный доступ к защищаемым данным.

Это называется *сериализацией*: потоки по очереди обращаются к данным, защищенным мьютексом, они должны получать к ним доступ не одновременно, а последовательно. Соответственно, чтобы допустить реальный конкурентный доступ, конструкция структуры данных должна быть хорошо продуманной. Одни структуры

данных лучше подходят для реального конкурентного доступа к ним, другие — хуже, но сама идея остается неизменной: чем меньше защищенная область, тем меньше операций подвергается сериализации и тем выше возможность конкурентных обращений к данным.

Прежде чем обратиться к конструкциям структур данных, уделите немного времени рассмотрению ряда простых рекомендаций и узнаем, на что следует обратить внимание при разработке структур данных для конкурентного доступа.

6.1.1. Рекомендации по разработке структур данных для конкурентного доступа

Как уже упоминалось, при разработке структур данных, допускающих конкурентный доступ, следует обратить внимание на два аспекта: гарантию *безопасности* обращений к данным и *обеспечение* настоящего конкурентного доступа к данным. Основы обеспечения потокобезопасности структуры данных рассматривались в главе 3.

- ❑ Нужно гарантировать, что ни один поток не сможет столкнуться с нарушением инвариантов структуры данных из-за действий другого потока.
- ❑ Следует воспрепятствовать возникновению состояния гонки, присущего интерфейсу структуры данных, предоставив функцию для завершенных операций, а не для их поэтапного выполнения.
- ❑ Нужно обратить внимание на поведение структуры данных при наличии исключений и обеспечить соблюдение инвариантов.
- ❑ Следует свести к минимуму вероятность взаимной блокировки при использовании структуры данных, ограничив область действия блокировок и по возможности исключив вложенные блокировки.

Прежде чем обдумывать любое из этих предложений, важно вспомнить о тех ограничениях, которые желательно наложить на действия пользователей структуры данных: если один поток обращается к структуре данных через конкретную функцию, то какие функции можно безопасно вызвать из других потоков?

Это вопрос первостепенной важности. Как правило, конструкторам и деструкторам требуется исключительный доступ к структуре данных, но обеспечение ее недоступности до тех пор, пока конструктор не завершит свою работу, или после того, как будет запущен деструктор, возлагается на пользователя. Если в структуре данных поддерживаются операция присваивания, функция `swap()` или копирующий конструктор, то разработчик структуры должен решить, насколько они безопасны при одновременном вызове вместе с другими операциями и не требуют ли они от пользователя обеспечить им исключительный доступ, при этом что большинство функций для работы со структурой данных можно вызвать из нескольких потоков одновременно без каких-либо проблем.

Второй по важности вопрос, требующий рассмотрения, — это возможность обеспечения действительно конкурентного доступа. Подробных рекомендаций на этот

счет у меня нет, вместо них есть перечень вопросов, на которые разработчик структуры данных должен ответить сам себе.

- ❑ Может ли область действия блокировок ограничиваться таким образом, чтобы некоторые части операции выполнялись за пределами блокировки?
- ❑ Можно ли разные части структуры данных защитить разными мьютексами?
- ❑ Всем ли операциям требуется одинаковый уровень защиты?
- ❑ Можно ли простым изменением структуры данных расширить возможности конкурентного доступа, не затрагивая при этом семантику операций?

В основе всех этих вопросов лежит одна идея: как свести к минимуму необходимый объем сериализации и обеспечить наивысший уровень истинной конкурентности. Зачастую структуры данных допускают конкурентный доступ нескольких потоков, которые просто считывают из них данные, а тот поток, который записывает в нее данные, должен иметь исключительный доступ. Такой режим доступа поддерживается за счет использования конструкций, подобных `std::shared_mutex`. Вскоре будет показано, что для структуры данных также зачастую характерны поддержка конкурентного доступа потоков, выполняющих разные операции, и сериализация доступа тех потоков, которые пытаются выполнять одну и ту же операцию.

В самой простой потокобезопасной структуре для защиты данных обычно используются мьютексы и блокировки. Как следует из материалов главы 3, с их помощью можно решить целый ряд проблем, к тому же довольно легко гарантировать, что в конкретный момент доступ к структуре данных получит только один поток. Чтобы облегчить вам разработку потокобезопасных структур данных, в этой главе основное внимание будет уделено именно структурам данных, основанным на применении блокировок, а разработке структур данных, допускающих конкурентные обращения и свободных от блокировок, будет посвящена глава 7.

6.2. Конкурентные структуры данных с блокировками

Разработка структур данных, допускающих конкурентные обращения и применяющих блокировки, сводится к обеспечению блокировки при обращении к данным нужных мьютексов и к тому, чтобы продолжительность удержания такой блокировки была минимальной. Когда структура защищена всего одним мьютексом, выполнить эту задачу нелегко. В главе 3 говорилось, что при этом нужно убедиться в невозможности доступа к данным вне защиты, обеспеченной блокировкой мьютекса, а также в отсутствие присущего интерфейсу состояния гонки. При использовании отдельных мьютексов для защиты отдельных частей структуры данных проблем еще больше, поскольку возникает также вероятность взаимных блокировок, если операциям над структурой данных требуется блокировка более чем одного мьютекса. Теперь конструкцию структуры данных с несколькими мьютексами приходится продумывать еще тщательнее, чем конструкцию с одним мьютексом.

В этом разделе для разработки нескольких простых структур данных с использованием для их защиты мьютексов и блокировок будут применяться рекомендации из раздела 6.1.1. В каждом случае будут изыскиваться возможности повышения уровня конкурентного доступа без ущерба для потокобезопасности структуры данных.

Начнем с рассмотрения реализации стека из главы 3, так как это одна из самых простых структур данных, в которой задействуется всего один мьютекс. Потокобезопасна ли она? И насколько далека от достижения истинной конкурентности?

6.2.1. Потокобезопасный стек, использующий блокировки

Код потокобезопасного стека из главы 3 воспроизведен в листинге 6.1. Замысел состоял в написании потокобезопасной структуры данных, похожей на `std::stack<>`, которая поддерживает помещение элементов данных в стек и их извлечение из него.

Листинг 6.1. Определение класса для потокобезопасного стека

```
#include <exception>
struct empty_stack: std::exception
{
    const char* what() const throw();
};
template<typename T>
class threadsafe_stack
{
private:
    std::stack<T> data;
    mutable std::mutex m;
public:
    threadsafe_stack(){}
    threadsafe_stack(const threadsafe_stack& other)
    {
        std::lock_guard<std::mutex> lock(other.m);
        data=other.data;
    }
    threadsafe_stack& operator=(const threadsafe_stack&) = delete;
    void push(T new_value)
    {
        std::lock_guard<std::mutex> lock(m);
        data.push(std::move(new_value)); ← ❶
    }
    std::shared_ptr<T> pop()
    {
        std::lock_guard<std::mutex> lock(m);
        if(data.empty()) throw empty_stack(); ← ❷
        std::shared_ptr<T> const res(
            std::make_shared<T>(std::move(data.top()))); ← ❸
        data.pop(); ← ❹
        return res; ← ❺
    }
}
```

```

void pop(T& value)
{
    std::lock_guard<std::mutex> lock(m);
    if(data.empty()) throw empty_stack();
    value=std::move(data.top());    ← ❸
    data.pop();                    ← ❹
}
bool empty() const
{
    std::lock_guard<std::mutex> lock(m);
    return data.empty();
}
};

```

Посмотрим, в какой степени в этом коде применяются изложенные ранее рекомендации.

Во-первых, можно увидеть, что элементарная потокобезопасность обеспечивается защитой каждой компонентной функции блокировкой мьютекса `m`. Тем самым обеспечивается возможность доступа к данным в конкретный момент времени со стороны только одного потока. Поэтому, если каждой компонентной функцией поддерживаются инварианты, увидеть нарушение какого-либо из них не сможет ни один поток.

Во-вторых, существует вероятность состояния гонки между функцией `empty()` и любой из функций `pop()`, но, поскольку код выполняет явную проверку на пустоту стека при удержании блокировки в функции `pop()`, состояние гонки проблем не вызывает. За счет непосредственного возвращения извлеченного элемента данных в виде части вызова функции `pop()` удастся избежать возможного состояния гонки, которое могло бы сложиться при использовании отдельных компонентных функций `top()` и `pop()` наподобие тех, что имеются в определении класса `std::stack<>`.

В-третьих, существует несколько возможных источников исключений. Блокировка мьютекса может выдать исключение, но подобное случается крайне редко, свидетельствуя о проблемах с мьютексом или нехватке системных ресурсов, и к тому же является самой первой операцией в каждой компонентной функции. Поскольку при этом изменения в данные не вносятся, опасности не возникает. Снятие блокировки с мьютекса не может вызвать сбой, поэтому считается абсолютно безопасной операцией, а применение `std::lock_guard<>` гарантирует, что мьютекс никогда не останется заблокированным.

При вызове `data.push()` ❶ исключение может быть выдано, либо если выдача исключения произошла при копировании или перемещении значения данных, либо если для расширения структуры данных, в которой они хранятся, не хватает свободной распределяемой памяти. В любом случае `std::stack<>` гарантирует безопасность, поэтому проблем не возникнет.

В первом переопределении функции `pop()` исключение `empty_stack` может выдать сам код ❷, но это не повлечет за собой никаких изменений и операция будет безопасной. Исключение может быть выдано и при создании объекта `res` ❸ по двум причинам: исключение, выданное `std::make_shared` из-за невозможности распределения памяти под новый объект и внутренние данные, необходимые для

подсчета ссылок, или же исключение, выданное конструктором копирования или конструктором перемещения элемента возвращаемых данных при копировании или перемещении в только что распределенную область памяти. В обоих случаях среда выполнения C++ и стандартная библиотека гарантируют отсутствие утечек памяти и корректное уничтожение нового объекта, если он будет создан. Поскольку стек, где хранятся данные, пока еще не был изменен, волноваться не о чем. Вызов `data.pop()` ❹ не выдаст исключений и не возвратит результат, поэтому переопределение функции `pop()` в отношении выдачи исключений опасности не представляет.

Второе переопределение функции `pop()` практически аналогично первому, за исключением того, что на этот раз исключение может быть выдано оператором присваивания копированием или оператором присваивания перемещением ❺, а не конструктором нового объекта и экземпляром `std::shared_ptr`. И в этом случае до вызова `data.pop()` ❻ в структуру данных не вносятся никаких изменений, что по-прежнему гарантирует невыдачу исключений, следовательно, это переопределение в отношении выдачи исключений никакой опасности не представляет.

Наконец, функция `empty()` не изменяет никаких данных и также безопасна в отношении выдачи исключений.

Здесь есть два возможных источника возникновения взаимной блокировки из-за вызова пользовательского кода при удержании блокировки: конструктор копирования или конструктор перемещения (❶, ❸) и оператор присваивания копированием или присваивания перемещением ❹, применяемые к содержащимся в структуре элементам данных. Если эти функции либо вызывают компонентные функции в отношении стека, в который добавляется или из которого извлекается элемент, либо требуют блокировки любого типа, а при вызове компонентной функции стека удерживалась другая блокировка, появляется возможность взаимоблокировки. Но возложить ответственность за предотвращение подобных случаев лучше на пользователей стека, поскольку неразумно было бы ожидать, что добавление элемента в стек или его извлечение из стека обойдется без его копирования или распределения памяти под него.

Поскольку для защиты данных всеми компонентными функциями используется `std::lock_guard<>`, компонентные функции стека можно безопасно вызывать из любого количества потоков. Единственными небезопасными компонентными функциями являются конструкторы и деструкторы, но это не проблема, поскольку объект может быть и сконструирован, и уничтожен только один раз. Вызов компонентных функций для не до конца сконструированного или частично уничтоженного объекта ни к чему хорошему не приводит независимо от того, насколько правильно он завершится. Следовательно, пользователь должен гарантировать, что у других потоков не будет возможности обратиться к стеку, пока он не будет полностью сконструирован, и что все потоки станут порождать доступ к стеку до его уничтожения.

Хотя одновременный вызов компонентных функций из нескольких потоков благодаря использованию блокировок считается безопасным, в конкретный момент времени со структурой данных работает только один поток. Если наблюдается серьезная конкуренция при доступе к стеку, *сериялизация* потоков может снизить производительность приложения, поскольку за время ожидания снятия блокировки

поток не выполняет никакой полезной работы. Кроме того, стек не предоставляет каких-либо средств ожидания добавляемого элемента, поэтому, если потоку приходится его дожидаться, он должен периодически вызывать функцию `empty()` или вызывать функцию `pop()` и перехватывать исключение `empty_stack`. Если понадобится такой сценарий, то данную реализацию стека удачной не назовешь: тут или ожидающий поток будет потреблять ценные ресурсы, или пользователь обязан будет создать внешний код ожидания и уведомления, используя, к примеру, условные переменные, что сделает внутреннюю блокировку ненужной и неоправданно расточительной. Способ включения этого ожидания в саму структуру данных с применением условной переменной был показан на примере очереди из главы 4, поэтому его и рассмотрим.

6.2.2. Потокобезопасная очередь с использованием блокировок и условных переменных

Потокобезопасная очередь из главы 4 воспроизведена в листинге 6.2. Стек был построен по образцу `std::stack<>`, а очередь — по образцу `std::queue<>`. И тут опять интерфейс отличается от стандартного адантера контейнера из-за ограничений, связанных с созданием структуры данных, безопасной для конкурентного доступа к ней со стороны сразу нескольких потоков.

Листинг 6.2. Полное определение класса для потокобезопасной очереди с использованием условных переменных

```
template<typename T>
class threadsafe_queue
{
private:
    mutable std::mutex mut;
    std::queue<T> data_queue;
    std::condition_variable data_cond;
public:
    threadsafe_queue()
    {}
    void push(T new_value)
    {
        std::lock_guard<std::mutex> lk(mut);
        data_queue.push(std::move(new_value));
        data_cond.notify_one(); ← ❶
    }
    void wait_and_pop(T& value) ← ❷
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk, [this]{return !data_queue.empty();});
        value=std::move(data_queue.front());
        data_queue.pop();
    }
    std::shared_ptr<T> wait_and_pop() ← ❸
};
```



```

{
    std::unique_lock<std::mutex> lk(mut);
    data_cond.wait(lk, [this]{return !data_queue.empty();}); ←❶
    std::shared_ptr<T> res(
        std::make_shared<T>(std::move(data_queue.front())));
    data_queue.pop();
    return res;
}
bool try_pop(T& value)
{
    std::lock_guard<std::mutex> lk(mut);
    if(data_queue.empty())
        return false;
    value=std::move(data_queue.front());
    data_queue.pop();
    return true;
}
std::shared_ptr<T> try_pop()
{
    std::lock_guard<std::mutex> lk(mut);
    if(data_queue.empty())
        return std::shared_ptr<T>(); ←❷
    std::shared_ptr<T> res(
        std::make_shared<T>(std::move(data_queue.front())));
    data_queue.pop();
    return res;
}
bool empty() const
{
    std::lock_guard<std::mutex> lk(mut);
    return data_queue.empty();
}
};

```

Структура реализации очереди, показанная с листинге 6.2, похожа на стек из листинга 6.1, за исключением вызова `data_cond.notify_one()` в `push()` ❶ и функции `wait_and_pop()` (❷ и ❸). Два переопределения `try_pop()` почти идентичны функциям `pop()` из листинга 6.1, но здесь они не выдают исключение, если очередь пуста. Вместо этого они возвращают либо `bool`-значение, показывающее, было ли извлечено значение, либо в случае с переопределением, возвращающим указатель ❸, — `NULL`-указатель, если нечего извлекать. Точно так же можно было бы реализовать и стек. Если неключить функции `wait_and_pop()`, то анализ, проделанный для стека, применим и здесь.

Новые функции `wait_and_pop()` позволяют решить проблему ожидания поступления значения в очередь, с которой мы уже сталкивались, когда рассматривали стек: вместо периодического вызова `empty()` ожидающий поток может вызвать `wait_and_pop()`, а структура данных обработает ожидание с помощью условной переменной. Возвращения из вызова `data_cond.wait()` не будет, пока очередь содержит хотя бы один элемент, поэтому в данном месте программы беспокоиться за вероятность пустой очереди не придется и данные по-прежнему защищены блокировкой

мьютекса. Следовательно, эти функции не добавляют новых состояний гонки или вероятности взаимной блокировки, обеспечивая сохранность инвариантов.

А вот относительно безопасности при выдаче исключений есть неприятный нюанс: если в состоянии ожидания находится более одного потока, то при поступлении элемента в очередь на вызов `data_cond.notify_one()` среагирует только один поток. Но если затем этот поток выдаст исключение в `wait_and_pop()`, например при конструировании нового объекта `std::shared_ptr<>` ❹, то другие потоки разбуджены не будут. Если это неприемлемо, данный вызов можно легко заменить вызовом `data_cond.notify_all()`, который разбудит все потоки, но большинство из них тут же снова упадет в спячку, как только обнаружится, что очередь опять опустела. Второй вариант заключается в наличии в `wait_and_pop()` в случае выдачи исключения вызова `notify_one()`, что позволит другому потоку попытаться извлечь сохраненное значение. Третий вариант предусматривает перемещение инициализации `std::shared_ptr<>` в вызов `push()` и сохранение экземпляров `std::shared_ptr<>`, а не непосредственных значений данных. Тогда копирование `std::shared_ptr<>` вне внутренней очереди `std::queue<>` не сможет выдать исключение и `wait_and_pop()` снова станет безопасной. В листинге 6.3 показана реализация очереди, переработанная с учетом этих соображений.

Листинг 6.3. Потокобезопасная очередь, содержащая экземпляры `std::shared_ptr<>`

```
template<typename T>
class threadsafe_queue
{
private:
    mutable std::mutex mut;
    std::queue<std::shared_ptr<T> > data_queue;
    std::condition_variable data_cond;
public:
    threadsafe_queue()
    {}
    void wait_and_pop(T& value)
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk, [this]{return !data_queue.empty();});
        value=std::move(*data_queue.front()); ←❶
        data_queue.pop();
    }
    bool try_pop(T& value)
    {
        std::lock_guard<std::mutex> lk(mut);
        if(data_queue.empty())
            return false;
        value=std::move(*data_queue.front()); ←❷
        data_queue.pop();
        return true;
    }
    std::shared_ptr<T> wait_and_pop()
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk, [this]{return !data_queue.empty();});
        std::shared_ptr<T> res=data_queue.front(); ←❸
    }
};
```

```

        data_queue.pop();
        return res;
    }
    std::shared_ptr<T> try_pop()
    {
        std::lock_guard<std::mutex> lk(mut);
        if(data_queue.empty())
            return std::shared_ptr<T>();
        std::shared_ptr<T> res=data_queue.front(); ←❶
        data_queue.pop();
        return res;
    }
    void push(T new_value)
    {
        std::shared_ptr<T> data(
            std::make_shared<T>(std::move(new_value))); ←❷
        std::lock_guard<std::mutex> lk(mut);
        data_queue.push(data);
        data_cond.notify_one();
    }
    bool empty() const
    {
        std::lock_guard<std::mutex> lk(mut);
        return data_queue.empty();
    }
};

```

Основные последствия хранения данных, обернутых в `std::shared_ptr<>`, просты и понятны: `pop`-функции, принимающие ссылку на переменную для получения нового значения, должны теперь разыменовать сохраненный указатель (❶ и ❷), а `pop`-функции, возвращающие экземпляр `std::shared_ptr<>`, могут извлечь его из очереди (❸ и ❹), перед возвращением вызывающему коду.

В сохранении данных с помощью `std::shared_ptr<>` есть еще одно преимущество: распределение памяти под новый экземпляр теперь может проводиться за пределами блокировки в функции `push()` ❸, а в коде листинга 6.2 его приходилось выполнять при удержании блокировки в функции `pop()`. Поскольку обычно операция распределения памяти сопряжена с немалыми затратами, это обстоятельство может пойти на пользу производительности очереди за счет сокращения времени удержания мьютекса, что позволит другим потокам в высвободившемся промежутке времени выполнять операции над очередью.

Точно так же, как и в примере со стеком, использование мьютекса для защиты всей структуры данных ограничивает поддержку этой очереди конкурентности. Хотя на очереди в различных компонентных функциях могут быть заблокированы сразу несколько потоков, в конкретный момент выполнять какую-либо работу может только один из них. Отчасти это ограничение обусловлено тем, что в реализации задействуется `std::queue<>`: теперь за счет применения стандартного контейнера вы располагаете одним элементом данных, который либо защищен, либо нет. Если взять под контроль подробности реализации структуры данных, можно обеспечить более подробную детализацию блокировки и поднять возможности конкурентности на более высокий уровень.

6.2.3. Потокобезопасная очередь с использованием подробной детализации блокировок и условных переменных

В коде листингов 6.2 и 6.3 мы имели дело с одним защищенным элементом данных (`data_queue`), а следовательно, с одним мьютексом. Чтобы воспользоваться более подробной детализацией блокировки, нужно проникнуть в саму очередь, в ее составляющие, и связать по одному мьютексу с каждым отдельно взятым элементом данных.

Самой простой структурой данных для очереди является односвязный список (рис. 6.1). Очередь состоит из `head` — указателя на первый элемент списка, и каждый элемент в ней указывает на следующий элемент. Элементы данных удаляются из очереди путем замены значения в указателе `head` указателем на следующий элемент с последующим возвращением данных, на которые ссылалось прежнее значение `head`.

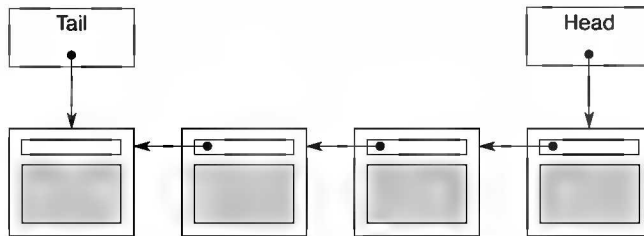


Рис. 6.1. Очередь, использующая односвязный список

Элементы добавляются в очередь с другого конца. Для этого у нее есть указатель `tail`, ссылающийся на последний элемент списка. Новые узлы добавляются изменением значения указателя `next` таким образом, чтобы он указывал на новый узел, и последующим обновлением указателя `tail`, чтобы он указывал на новый элемент. Когда список пуст, указатели `head` и `tail` имеют значение `NULL`.

В листинге 6.4 показана простая реализация очереди, созданной на основе урезанной версии интерфейса к очереди из листинга 6.2. В ней есть только одна функция `try_pop()` и нет функции `wait_and_pop()`, поскольку эта очередь поддерживает только однопоточное использование.

Листинг 6.4. Простая реализация однопоточной очереди

```
template<typename T>
class queue
{
private:
    struct node
    {
        T data;
        std::unique_ptr<node> next;
```



```

    node(T data_):
        data(std::move(data_))
    {}
};
std::unique_ptr<node> head; ← ❶
node* tail; ← ❷
public:
    queue(): tail(nullptr)
    {}
    queue(const queue& other)=delete;
    queue& operator=(const queue& other)=delete;
    std::shared_ptr<T> try_pop()
    {
        if(!head)
        {
            return std::shared_ptr<T>();
        }
        std::shared_ptr<T> const res(
            std::make_shared<T>(std::move(head->data)));
        std::unique_ptr<node> const old_head=std::move(head);
        head=std::move(old_head->next); ← ❸
        if(!head)
            tail=nullptr;
        return res;
    }
    void push(T new_value)
    {
        std::unique_ptr<node> p(new node(std::move(new_value)));
        node* const new_tail=p.get();
        if(tail)
        {
            tail->next=std::move(p); ← ❹
        }
        else
        {
            head=std::move(p); ← ❺
        }
        tail=new_tail; ← ❻
    }
};

```

Прежде всего заметьте, что в коде листинга 6.4 для управления узлами задействовуются указатели `std::unique_ptr<node>`, тем самым гарантируется, что они (и данные, на которые они ссылаются) будут удаляться, как только станут не нужны, и явно использовать функцию `delete` не потребуется. Цепочка владения управляется с `head`, а вот `tail` — всего лишь указатель на последний узел, так как он должен ссылаться на узел, уже принадлежащий `std::unique_ptr<node>`.

В однопоточном контексте эта реализация работает неплохо, однако попытка использовать более подробную детализацию в многопоточной среде выявит два проблемных места. При наличии двух элементов данных, `head` ❶ и `tail` ❷, можно воспользоваться двумя мьютексами, по одному для защиты `head` и `tail`, но тут как раз и проявятся две проблемы.

Наиболее явная будет выражаться в том, что функция `push()` может изменять как `head` ❸, так и `tail` ❷, следовательно, ей придется заблокировать оба мьютекса. Проблема, конечно, неприятная, но разрешимая, поскольку заблокировать оба мьютекса можно. А главная проблема состоит в том, что обе функции, `push()` и `pop()`, обращаются к имеющемуся в этом узле указателю `next`: `push()` обновляет `tail->next` ❹, а `try_pop()` считывает `head->next` ❸. Если в очереди всего один элемент, то `head==tail`, следовательно, `head->next` и `tail->next` — один и тот же объект, нуждающийся в защите. Поскольку, не прочитав `head` и `tail`, понять, что это один и тот же объект, невозможно, теперь нужно заблокировать один и тот же мьютекс как в `push()`, так и в `try_pop()`. Следовательно, ситуация не улучшилась. Можно ли решить эту дилемму?

Обеспечение конкурентности за счет разделения данных

Проблему можно решить, заранее разместив в очереди пустой фиктивный узел и тем самым обеспечив постоянное наличие в ней хотя бы одного узла, чтобы отделить узел, к которому обращаются в голове, от узла, к которому обращаются в хвосте. Теперь при пустой очереди как `head`, так и `tail` указывают на фиктивный узел и не содержат значение `NULL`. Это нас вполне устраивает, поскольку `try_pop()` не обращается к `head->next`, когда очередь пуста. Если к очереди добавляется узел (то есть в ней появляется один настоящий узел), `head` и `tail` указывают на разные узлы, поэтому состояния гонки при обращениях `head->next` и `tail->next` не возникает. Недостатком такого решения является то, что применение фиктивных узлов требует добавления дополнительного уровня косвенности для хранения данных по указателю. Новая реализация показана в листинге 6.5.

Листинг 6.5. Простая очередь с фиктивным узлом

```
template<typename T>
class queue
{
private:
    struct node
    {
        std::shared_ptr<T> data;    ←❶
        std::unique_ptr<node> next;
    };
    std::unique_ptr<node> head;
    node* tail;
public:
    queue():
        head(new node), tail(head.get()) ←❷
    {}
    queue(const queue& other)=delete;
    queue& operator=(const queue& other)=delete;
    std::shared_ptr<T> try_pop()
    {
        if(head.get()==tail) ←❸
        {
```

```

    return std::shared_ptr<T>();
}
std::shared_ptr<T> const res(head->data); ←①
std::unique_ptr<node> old_head=std::move(head);
head=std::move(old_head->next); ←⑤
return res; ←⑥
}
void push(T new_value)
{
    std::shared_ptr<T> new_data(
        std::make_shared<T>(std::move(new_value))); ←⑦
    std::unique_ptr<node> p(new node); ←⑧
    tail->data=new_data; ←⑨
    node* const new_tail=p.get();
    tail->next=std::move(p);
    tail=new_tail;
}
};

```

В `try_pop()` внесены минимальные изменения. Во-первых, `head` сравнивается с `tail` ③, а не со значением `NULL`, так как присутствие фиктивного узла означает, что у `head` никогда не будет значения `NULL`. Поскольку `head` относится к указателю `std::unique_ptr<node>`, для выполнения операции сравнения следует вызвать `head.get()`. Во-вторых, так как в `node` теперь хранится указатель на данные ①, этот указатель можно извлечь напрямую ④, избавившись от необходимости создавать новый экземпляр `T`. А вот функция `push()` изменилась довольно сильно. Сначала нужно создать в куче новый экземпляр `T` и получить его в собственность в `std::shared_ptr<T>` ② (обратите внимание на использование `std::make_shared` для того, чтобы исключить издержки от повторного распределения памяти под счетчик ссылок). Созданный новый узел станет новым фиктивным узлом, поэтому предоставлять `new_value` конструктору ⑧ не нужно. Вместо этого для старого фиктивного узла устанавливаются данные только что распределенной копии `new_value` ⑨. И наконец, чтобы существовал фиктивный узел, его нужно создать в конструкторе ②.

Наверняка вас интересует, что дают эти изменения и как они помогают сделать очередь потокобезопасной. Итак, `push()` теперь обращается только к `tail`, но не к `head`, что уже хорошо. Функция `try_pop()` обращается как к `head`, так и к `tail`, но `tail` требуется только для начального сравнения, следовательно, блокировка удерживается недолго. Главный выигрыш состоит в том, что при наличии фиктивного узла `try_pop()` и `push()` никогда не работают с одним и тем же узлом, следовательно, мьютекс, охватывающий всю очередь, больше не нужен. Можно иметь один мьютекс для `head` и один для `tail`. А куда же ставятся блокировки?

Наша цель — максимально раскрыть возможности для конкурентности, следовательно, продолжительность удержания блокировок хотелось бы свести к минимуму. С функцией `push()` все просто: мьютекс должен быть заблокирован при всех обращениях к `tail`, то есть он блокируется после распределения памяти под новый узел ⑧ и до присваивания данных текущему хвостовому узлу ⑨. Затем блокировку нужно удерживать, пока функция не будет выполнена до конца.

С функцией `try_pop()` дела обстоят сложнее. Сперва нужно заблокировать мьютекс на обращении к `head` и удерживать его, пока работа с `head` не будет закончена. Этим мьютексом определяется, какой поток извлекает данные, поэтому блокировку требуется выполнить в первую очередь. После изменения значения `head` ❸ мьютекс можно разблокировать: в момент возвращения результата ❹ он уже не нужен. Остается только выяснить, необходима ли блокировка хвостового мьютекса при обращении к `tail`. Поскольку обращаться к `tail` придется всего один раз, мьютекс можно заблокировать только на время чтения. Лучшее всего сделать это, заключив его в функцию. Так как код, которому требуется мьютекс, заблокированный на обращении к `head`, фактически является всего лишь частью компонентного кода, понятно, что и его тоже лучше заключить в функцию. Окончательная версия кода показана в листинге 6.6.

Листинг 6.6. Потокбезопасная очередь с более подробной детализацией блокировок

```
template<typename T>
class threadsafe_queue
{
private:
    struct node
    {
        std::shared_ptr<T> data;
        std::unique_ptr<node> next;
    };
    std::mutex head_mutex;
    std::unique_ptr<node> head;
    std::mutex tail_mutex;
    node* tail;
    node* get_tail()
    {
        std::lock_guard<std::mutex> tail_lock(tail_mutex);
        return tail;
    }
    std::unique_ptr<node> pop_head()
    {
        std::lock_guard<std::mutex> head_lock(head_mutex);

        if(head.get()==get_tail())
        {
            return nullptr;
        }
        std::unique_ptr<node> old_head=std::move(head);
        head=std::move(old_head->next);
        return old_head;
    }
public:
    threadsafe_queue():
        head(new node),tail(head.get())
    {}
    threadsafe_queue(const threadsafe_queue& other)=delete;
    threadsafe_queue& operator=(const threadsafe_queue& other)=delete;
```

```

std::shared_ptr<T> try_pop()
{
    std::unique_ptr<node> old_head=pop_head();
    return old_head?old_head->data:std::shared_ptr<T>();
}
void push(T new_value)
{
    std::shared_ptr<T> new_data(
        std::make_shared<T>(std::move(new_value)));
    std::unique_ptr<node> p(new node);
    node* const new_tail=p.get();
    std::lock_guard<std::mutex> tail_lock(tail_mutex);
    tail->data=new_data;
    tail->next=std::move(p);
    tail=new_tail;
}
};

```

Оценим этот код с точки зрения его соответствия рекомендациям, приведенным в подразделе 6.1.1. Прежде чем выискивать нарушения инвариантов, надо определить, что они сводятся к следующим формулировкам:

- ☐ `tail->next==nullptr`;
- ☐ `tail->data==nullptr`;
- ☐ `head==tail` означает пустой список;
- ☐ у списка с одним элементом `head->next==tail`;
- ☐ для каждого имеющегося в списке узла `x`, где `x!=tail`, `x->data` указывает на экземпляр `T`, а `x->next` — на следующий узел в списке. Если `x->next==tail`, то `x` — последний узел списка;
- ☐ следуя по `next`-узлам от `head`, в конечном счете можно оказаться в `tail`.

Сама по себе функция `push()` проста и понятна: мьютекс `tail_mutex` защищает только изменения в структуре данных, и инвариант ими сохраняется, поскольку новый хвостовой узел является пустым, а указатели `data` и `next` для старого хвостового узла, который теперь является последним настоящим узлом списка, установлены правильно.

Все самое интересное находится в функции `try_pop()`. Оказывается, блокировка мьютекса `tail_mutex` нужна не только для защиты чтения самого указателя `tail`, но и для гарантии того, что состояние гонки не возникнет при чтении данных из `head`. При отсутствии этого мьютекса высока вероятность того, что одновременно одним потоком будет вызвана функция `try_pop()`, а другим — функция `push()`, а порядок выполнения их операций окажется не определен. Хотя блокировка мьютекса удерживается каждой компонентной функцией или удерживаются *разные* мьютексы, у них появляется возможность обращения к одним и тем же данным, ведь все данные в очереди появляются благодаря вызову функции `push()`. Из-за того что потоки могут обращаться к одним и тем же данным, не определяя конкретный порядок, возникнет, как было показано в главе 5, состояние гонки, сопряженное с неопределенным поведением. Благодаря блокировке мьютекса `tail_mutex` в функции

`get_tail()` решаются все проблемы. Поскольку вызов `get_tail()` блокирует тот же самый мьютекс, который блокирует и вызов `push()`, между двумя вызовами определяется порядок. Вызов `get_tail()` происходит либо до вызова `push()`, и в таком случае код этой функции видит старое значение `tail`, либо после вызова `push()`, и в таком случае код видит новое значение `tail` и новые данные, прикрепленные к предыдущему значению `tail`.

Важно также, что вызов `get_tail()` происходит во время блокировки мьютекса `head_mutex`. В противном случае вызов `pop_head()` мог бы застрять между вызовом `get_tail()` и блокировкой мьютекса `head_mutex`, из-за того что другие потоки вызвали `try_pop()` (а следовательно, и `pop_head()`) и захватили блокировку первыми, не давая выполнять работу исходному потоку:

```
std::unique_ptr<node> pop_head() ← Неправильная реализация
{
    node* const old_tail=get_tail();
    std::lock_guard<std::mutex> head_lock(head_mutex);

    if(head.get()==old_tail) ← ❷
    {
        return nullptr;
    }
    std::unique_ptr<node> old_head=std::move(head);
    head=std::move(old_head->next); ← ❸
    return old_head;
}
```

❶ Получение старого значения tail вне блокировки head_mutex

В этом неправильном сценарии, где вызов `get_tail()` ❶ выполнен вне области действия блокировки, может обнаружиться, что и `head`, и `tail` изменились к тому времени, когда ваш исходный поток смог заблокировать мьютекс `head_mutex`, и возвращенный узел `tail` не только перестал быть хвостовым, но даже уже не является частью списка. Как следствие, сравнение `head` с `old_tail` ❷ даст сбой, даже если `head` является последним узлом. Поэтому при обновлении `head` ❸ у вас в итоге может получиться, что `head` переместится за `tail` и окажется за концом списка, разрушив структуру данных. В правильной реализации из листинга 6.6 вызов `get_tail()` выполняется под блокировкой мьютекса `head_mutex`. Тем самым гарантируется, что никакие другие потоки не могут изменить `head`, и `tail` будет только продвигаться дальше по списку по мере добавления новых узлов в вызовах функции `push()`, соблюдая абсолютную безопасность. Указатель `head` никогда не сможет перейти за значение, возвращенное вызовом `get_tail()`, поэтому инварианты не будут нарушены.

Как только за счет обновления `head` функция `pop_head()` удалит узел из очереди, мьютекс будет разблокирован и `try_pop()` сможет извлечь данные и удалить узел, если он существовал (или вернуть экземпляр `std::shared_ptr<>` со значением `NULL`, если его не было), соблюдая безопасность в расчете на то, что доступ к данному узлу можно получить лишь из одного потока.

Внешний интерфейс является подмножеством интерфейса из листинга 6.2, следовательно, к нему применим прежний анализ: у него нет склонности к состоянию гонки.

Интереснее будет разобраться с исключениями. Поскольку схемы распределения памяти под данные изменились, исключения теперь могут выдаваться в разных местах. Единственными операциями в `try_pop()`, способными выдавать исключения, являются блокировки мьютексов, и данные не изменяются, пока блокировки не будут установлены. Поэтому в отношении выдачи исключений функция `try_pop()` безопасна. В то же время функция `push()` распределяет в куче память под новые экземпляры `T` и `node`, и при каждом из этих распределений может быть выдано исключение. Но оба объекта, под которые только что была распределена память, присваиваются интеллектуальным указателям, и при выдаче исключения эта память будет освобождена. После установки блокировки никакие другие операции в `push()` не могут выдать исключение, поэтому вам опять не о чем беспокоиться и функция `push()` в отношении выдачи исключений также безопасна.

Поскольку изменения в интерфейс не вносились, новые внешние возможности для взаимной блокировки не возникали. Нет и никаких внутренних возможностей. Единственным местом установки двух блокировок является функция `pop_head()`, которая всегда блокирует `head_mutex`, а затем `tail_mutex`, поэтому взаимная блокировка никогда не возникает.

Остаются только вопросы, касающиеся возможности конкурентности. У этой структуры данных существенно более широкие возможности предоставления такого доступа, чем у структуры из листинга 6.2, поскольку у блокировок более подробная детализация и гораздо больше работы выполняется за их пределами. Например, в функции `push()` распределение памяти под новый узел и новый элемент данных происходит без удержания блокировки. Следовательно, его могут без проблем выполнять одновременно несколько потоков. В конкретный момент времени добавить свой новый узел в список может только один поток, но код для этого представляет собой всего лишь несколько присваиваний значений указателям, поэтому блокировка удерживается совсем недолго, если сравнивать с реализацией на основе применения `std::queue<>`, где она удерживается на протяжении всех операций распределения памяти внутри `std::queue<>`.

К тому же функция `try_pop()` удерживает блокировку `tail_mutex` только на время, необходимое для защиты считывания данных из указателя `tail`. Следовательно, почти все, что делается внутри вызова `try_pop()`, может происходить одновременно с вызовом функции `push()`. Кроме того, при удержании блокировки `head_mutex` выполняется минимальный объем операций, затратная операция `delete` (в деструкторе указателя `node`) протекает вне блокировки. За счет этого будет увеличиваться количество возможных конкурентных вызовов функции `try_pop()`: в конкретный момент вызвать функцию `pop_head()` может только один поток, но затем удалить свои старые узлы и безопасно вернуть данные могут сразу несколько потоков.

Ожидание поступления элемента

Итак, код в листинге 6.6 предоставляет потокобезопасную очередь с более подробной детализацией блокировок, но он поддерживает только функцию `try_pop()` без каких-либо ее переопределений. А как насчет поддержки весьма удобных функций `wait_and_pop()` из листинга 6.2? Можно ли реализовать идентичный интерфейс с применением более подробной детализации блокировок?

Ответ положительный, но возникает вопрос, как это сделать. Внести изменения в `push()` не составит труда: нужно лишь, как в листинге 6.2, добавить в конец функции вызов `data_cond.notify_one()`. Но не все так просто. Из-за необходимости получить максимум возможностей конкурентности к структуре данных мы воспользовались более подробной детализацией блокировок. Если блокировка мьютекса будет удерживаться в течение всего времени вызова `notify_one()`, как в листинге 6.2, то при пробуждении уведомляемого потока до снятия блокировки с мьютекса ему придется дожидаться этого снятия. В то же время если снять блокировку мьютекса до вызова `notify_one()`, то ожидающий поток может заблокировать этот мьютекс сразу же после пробуждения (при условии, что его не опередит какой-нибудь другой поток). Усовершенствование получится незначительным, но в некоторых случаях оно может сыграть весьма важную роль.

С функцией `wait_and_pop()` дело обстоит несколько сложнее, поскольку нужно решить, где будет ожидание, каким — предикат и какой мьютекс следует заблокировать. Будет ожидаться выполнение условия «очередь не пуста», которое можно представить в виде выражения `head!=tail`. При такой формулировке условия потребуется блокировка обоих мьютексов, `head_mutex` и `tail_mutex`, но при рассмотрении кода листинга 6.6 уже было решено, что заблокировать `tail_mutex` нужно не для сравнения как такового, а только для чтения `tail`, поэтому аналогичную логику можно применить и здесь. Если создать предикат вида `head!=get_tail()`, потребуется только удерживать блокировку мьютекса `head_mutex`, и ею можно воспользоваться для вызова `data_cond.wait()`. После добавления логики ожидания вся остальная реализация будет такой же, как и у функции `try_pop()`.

Второе переопределение `try_pop()` и соответствующее переопределение `wait_and_pop()` нужно будет тщательно продумать. Если заменить возвращение указателя `std::shared_ptr<>`, извлеченного из `old_head`, присваиванием методом копирования параметра `value`, то возможны проблемы с безопасностью выдачи исключений. К этому моменту элемент данных уже был удален из очереди и мьютекс разблокирован, осталось только вернуть данные вызывающему коду. Но если присваивание копированием выдаст исключение (что вполне возможно), элемент данных будет утрачен, поскольку вернуть его в то же место в очереди уже не получится.

Если фактический тип `T`, используемый в качестве аргумента шаблона, обладает не выдающим исключений оператором присваивания методом перемещения или не выдающей исключений операцией обмена (`swap`), то можно воспользоваться и этим вариантом, но предпочтительнее было бы обобщенное решение, подходящее для любого типа `T`. В данном случае, прежде чем удалять узел из списка, придется перемещать потенциально способную на выдачу исключения операцию в область действия блокировки.

Значит, понадобится еще одно переопределение `pop_head()`, извлекающее сохраненное значение для внесения изменения в список.

А вот функция `empty()` сравнительно проста: блокировка `head_mutex` и проверка условия `head==get_tail()` (см. код листинга 6.10). Код очереди в окончательном виде показан в листингах 6.7–6.10.

Листинг 6.7. Потокобезопасная очередь с блокировками и ожиданием: внутренний код и интерфейс

```
template<typename T>
class threadsafe_queue
{
private:
    struct node
    {
        std::shared_ptr<T> data;
        std::unique_ptr<node> next;
    };
    std::mutex head_mutex;
    std::unique_ptr<node> head;
    std::mutex tail_mutex;
    node* tail;
    std::condition_variable data_cond;
public:
    threadsafe_queue():
        head(new node), tail(head.get())
    {}
    threadsafe_queue(const threadsafe_queue& other)=delete;
    threadsafe_queue& operator=(const threadsafe_queue& other)=delete;
    std::shared_ptr<T> try_pop();
    bool try_pop(T& value);
    std::shared_ptr<T> wait_and_pop();
    void wait_and_pop(T& value);
    void push(T new_value);
    bool empty();
};
```

Помещение в очередь новых узлов выполняется довольно просто — реализация, показанная в листинге 6.8, близка к той, что была приведена ранее.

Листинг 6.8. Потокобезопасная очередь с блокировками и ожиданием: помещение в очередь новых значений

```
template<typename T>
void threadsafe_queue<T>::push(T new_value)
{
    std::shared_ptr<T> new_data(
        std::make_shared<T>(std::move(new_value)));
    std::unique_ptr<node> p(new node);
    {
        std::lock_guard<std::mutex> tail_lock(tail_mutex);
        tail->data=new_data;
        node* const new_tail=p.get();
        tail->next=std::move(p);
        tail=new_tail;
    }
    data_cond.notify_one();
}
```

Как уже упоминалось, вся сложность сосредоточена в `pop`-компоненте, использующем для упрощения ряд вспомогательных функций. В листинге 6.9 показана реализация функции `wait_and_pop()` и связанных с ней вспомогательных функций.

Листинг 6.9. Потокбезопасная очередь с блокировками и ожиданием: wait_and_pop()

```

template<typename T>
class threadsafe_queue
{
private:
    node* get_tail()
    {
        std::lock_guard<std::mutex> tail_lock(tail_mutex);
        return tail;
    }
    std::unique_ptr<node> pop_head() ←❶
    {
        std::unique_ptr<node> old_head=std::move(head);
        head=std::move(old_head->next);
        return old_head;
    }
    std::unique_lock<std::mutex> wait_for_data() ←❷
    {
        std::unique_lock<std::mutex> head_lock(head_mutex);
        data_cond.wait(head_lock,[&]{return head.get() != get_tail();});
        return std::move(head_lock); ←❸
    }
    std::unique_ptr<node> wait_pop_head()
    {
        std::unique_lock<std::mutex> head_lock(wait_for_data()); ←❹
        return pop_head();
    }
    std::unique_ptr<node> wait_pop_head(T& value)
    {
        std::unique_lock<std::mutex> head_lock(wait_for_data()); ←❺
        value=std::move(*head->data);
        return pop_head();
    }
public:
    std::shared_ptr<T> wait_and_pop()
    {
        std::unique_ptr<node> const old_head=wait_pop_head();
        return old_head->data;
    }
    void wait_and_pop(T& value)
    {
        std::unique_ptr<node> const old_head=wait_pop_head(value);
    }
}

```

Реализация pop-компонента, показанная в листинге 6.9, содержит несколько небольших вспомогательных функций, упрощающих код и устраняющих возможное дублирование: функцию pop_head() ❶, изменяющую список для удаления элемента head, и функцию wait_for_data() ❷, ожидающую появления в очереди каких-либо извлекаемых данных. Функция wait_for_data() заслуживает особого внимания, поскольку в ней не только задается ожидание на условной переменной с применением в качестве предиката лямбда-функции, но и возвращается экземпляр блокировки вызывающему коду ❸. Тем самым гарантируется, что при изменении данных соот-

ветствующим переопределением функции `wait_pop_head()` в строках кода ❹ и ❺ удерживается та же самая блокировка. Кроме того, функция `pop_head()` повторно используется в коде функции `try_pop()` (листинг 6.10).

Листинг 6.10. Потокобезопасная очередь с блокировками и ожиданием: `try_pop()` и `empty()`

```
template<typename T>
class threadsafe_queue
{
private:
    std::unique_ptr<node> try_pop_head()
    {
        std::lock_guard<std::mutex> head_lock(head_mutex);
        if(head.get()==get_tail())
        {
            return std::unique_ptr<node>();
        }
        return pop_head();
    }
    std::unique_ptr<node> try_pop_head(T& value)
    {
        std::lock_guard<std::mutex> head_lock(head_mutex);
        if(head.get()==get_tail())
        {
            return std::unique_ptr<node>();
        }
        value=std::move(*head->data);
        return pop_head();
    }
public:
    std::shared_ptr<T> try_pop()
    {
        std::unique_ptr<node> old_head=try_pop_head();
        return old_head?old_head->data:std::shared_ptr<T>();
    }
    bool try_pop(T& value)
    {
        std::unique_ptr<node> const old_head=try_pop_head(value);
        return old_head;
    }
    bool empty()
    {
        std::lock_guard<std::mutex> head_lock(head_mutex);
        return (head.get()==get_tail());
    }
};
```

Данная реализация очереди послужит основой для рассматриваемой в главе 7 очереди без применения блокировок. Речь идет о *неограниченной* очереди, в которую потоки могут и далее помещать новые значения, пока не закончится свободная память, даже если значения не будут удаляться. Альтернативой ей служит *ограниченная* очередь с фиксированной при ее создании максимальной длиной. Как только ограниченная очередь заполнится, попытки помещения в нее дополнительных элементов

либо будут давать сбой, либо станут блокироваться, пока при извлечении элемента не освободится место. Ограниченные очереди пригодятся для равномерного распределения работы между потоками на основе выполняемых задач (см. главу 8). Это не даст потоку или потокам, наполняющим очередь, сильно опережать работу потока или потоков, считывающих элементы из очереди.

Показанную в книге неограниченную очередь можно легко превратить в очередь ограниченной длины, воспользовавшись ожиданием на условной переменной в функции `push()`. Вместо ожидания появления в очереди элементов, как это сделано в функции `pop()`, придется ждать, когда количество элементов в ней станет меньше максимально допустимого. Дальнейшие рассуждения по поводу ограниченных очередей выходят за рамки книги, поэтому оставим эту тему и перейдем к более сложным структурам данных.

6.3. Разработка более сложных структур данных с использованием блокировок

Стеки и очереди не отличаются особой сложностью: их интерфейс крайне прост, а предназначение — узконаправленное. Но подобной простотой отличаются далеко не все структуры данных, большинство из них поддерживает весьма широкий диапазон операций. В принципе, это может расширить возможность конкурентности, но при этом значительно усложнится защита данных, поскольку нужно будет учесть схемы множественного доступа к ним. При разработке конкурентных структур данных важно учесть характер выполняемых операций.

Чтобы понять суть возникающих проблем, рассмотрим структуру поисковой таблицы.

6.3.1. Создание потокобезопасной поисковой таблицы с использованием блокировок

В поисковой таблице, или в словаре, значения одного типа (ключевого) связаны со значениями того же или другого типа (отображаемого). В общем, суть такой структуры заключается в том, чтобы позволить коду запрашивать данные, связанные с заданным ключом. В стандартной библиотеке C++ такая возможность предоставляется с помощью ассоциативных контейнеров `std::map<>`, `std::multimap<>`, `std::unordered_map<>` и `std::unordered_multimap<>`.

Принцип использования поисковой таблицы не такой, как у стека или очереди, где практически каждая операция изменяет их тем или иным образом, либо добавляя, либо удаляя элемент. Поисковая таблица может изменяться крайне редко. Одним из примеров такого сценария служит простой DNS-кэш из листинга 3.13, который представляется сильно усеченным интерфейсом по сравнению с `std::map<>`. Как было показано на примере стека и очереди, интерфейсы стандартных контейнеров не подходят для тех случаев, когда от структуры данных требуется обеспечить конкурентность сразу нескольким потокам. Конструкциям их интерфейсов присущи состояния гонки, поэтому их нужно урезать и пересматривать.

Самой серьезной проблемой интерфейса `std::map<>` с точки зрения конкурентности являются итераторы. Хотя можно обзавестись итератором, предоставляющим безопасный доступ к контейнеру даже при условии, что другие потоки могут обращаться к контейнеру и вносить в него изменения, легкой эту задачу не назовешь. Корректно управляемые итераторы требуют умения справиться, к примеру, с такой проблемой, когда другой поток удаляет элемент, на который ссылается итератор. Ее решение может оказаться довольно сложным. На первый взгляд, создавая интерфейс потокобезопасной поисковой таблицы, итераторы нужно выбросить. При условии весьма серьезной привязки интерфейса `std::map<>` и других ассоциативных контейнеров стандартной библиотеки к итераторам лучше будет, наверное, оставить их и разработать интерфейс с нуля.

Для поисковой таблицы понадобится всего несколько основных операций.

- ❑ Добавление новой пары «ключ — значение».
- ❑ Изменение значения, связанного с заданным ключом.
- ❑ Удаление ключа и связанного с ним значения.
- ❑ Получение значения, связанного с заданным ключом, если таковой имеется.

Есть также несколько операций, выполняемых над всем контейнером, из которых можно извлечь пользу, например проверка контейнера на пустоту, получение мгновенного образа полного списка ключей или полного набора пар «ключ — значение».

Если придерживаться простых рекомендаций по обеспечению потокобезопасности, например не возвращать ссылки и накладывать простую блокировку мьютекса целиком на всю компонентную функцию, то все эти операции будут безопасны: они станут выполняться либо до изменения, реализуемого другим потоком, либо после него. Наибольшая вероятность состояния гонки возникает при добавлении новой пары «ключ — значение»: если два потока добавляют новое значение, то только один из них будет первым, следовательно, второй со своей задачей не справится. В качестве одного из решений можно свести добавление и изменение в одну компонентную функцию, как было сделано для DNS-кэша в коде листинга 3.13.

С точки зрения интерфейса остается лишь один интересный момент, касающийся получения связанного значения, где о наличии ключа говорится: «Если таковой имеется». Один из вариантов — позволить пользователю предоставить результат по умолчанию, возвращаемый при отсутствии ключа:

```
mapped_type get_value(key_type const& key, mapped_type default_value);
```

В таком случае, если значение по умолчанию `default_value` не было предоставлено явным образом, можно воспользоваться созданным заранее для случая по умолчанию экземпляром `mapped_type`. Можно пойти еще дальше и вернуть вместо простого экземпляра `mapped_type` объект `std::pair<mapped_type, bool>`, где `bool` показывает, было ли значение. Еще один вариант — вернуть интеллектуальный указатель на значение: если значение указателя `NULL`, то значения для возвращения не было.

Как уже упоминалось, после принятия решения о конструкции интерфейса (при условии, что интерфейсу не свойственно состояние гонки) потокобезопасность можно гарантировать использованием единственного мьютекса и простой блокировки,

устанавливаемой в каждой компонентной функции для защиты исходной структуры данных. По тем самым будут утеряны предоставляемые отдельными функциями возможности конкурентности для чтения и изменения структуры данных. Один из вариантов решения этой проблемы заключается в применении мьютекса, поддерживающего несколько читающих потоков или один записывающий поток, например `std::shared_mutex`, который задействован в коде листинга 3.13. Разумеется, такой подход расширит возможности конкурентности, но изменять структуру данных в конкретный момент времени сможет только один поток. В идеале хотелось бы добиться более впечатляющего результата.

Разработка отображаемой структуры данных для подробной детализации блокировок

Как и в случае с очередью, рассмотренной в подразделе 6.2.3, чтобы допустить изменение подробно детализированной блокировки, нужно тщательно продумать все детали структуры данных, а не заключать ее в уже существующий контейнер, например в `std::map<>`. Реализовать ассоциативный контейнер наподобие нашей поисковой таблицы можно тремя широко используемыми способами:

- ❑ в виде бинарного дерева, такого как красно-черное дерево;
- ❑ в виде отсортированного массива;
- ❑ в виде хеш-таблицы.

Бинарное дерево не предоставляет слишком большого пространства для расширения возможностей конкурентности: каждый поиск или изменение должны начинаться с обращения к корневому узлу, который по этой причине следует заблокировать. Блокировку можно снять, как только обращающийся поток спустится вниз по дереву, однако это ненамного лучше единой блокировки, распространенной на всю структуру данных.

Отсортированный массив еще хуже, поскольку заранее невозможно сказать, в каком месте массива находятся искомые данные, поэтому нужна единая блокировка на весь массив.

Остается хеш-таблица. При фиксированном количестве сегментов вопрос принадлежности ключа сегменту — исключительно свойство ключа и его хеш-функции. Это означает, что можно применять безопасную отдельную блокировку каждого сегмента. При повторном использовании мьютекса, который поддерживает несколько считывающих потоков или один записывающий поток, возможность конкурентности увеличивается в N раз, где N — количество сегментов. Недостатком является необходимость задействования для ключа хорошей хеш-функции. Стандартная библиотека C++ предоставляет шаблон `std::hash<>`, которым можно воспользоваться для этой цели. Он уже приспособлен к основным типам, таким как `int`, и к распространенным библиотечным типам, таким как `std::string`, а пользователь может легко приспособить его и для других типов ключей. Если принять как пример стандартные неупорядоченные контейнеры и взять в качестве параметра шаблона, который будет использоваться для хеширования, тип функционального объекта, пользователь может выбрать, следует ли приспособливать `std::hash<>` для его типа ключа или же предоставить отдельную хеш-функцию.

Взглянем на код. На что может быть похожа реализация потокобезопасной поисковой таблицы? Один из возможных вариантов показан в листинге 6.11.

Листинг 6.11. Потокобезопасная поисковая таблица

```
template<typename Key,typename Value,typename Hash=std::hash<Key> >
class threadsafe_lookup_table
{
private:
    class bucket_type
    {
    private:
        typedef std::pair<Key,Value> bucket_value;
        typedef std::list<bucket_value> bucket_data;
        typedef typename bucket_data::iterator bucket_iterator;
        bucket_data data;
        mutable std::shared_mutex mutex;          ←❶

        bucket_iterator find_entry_for(Key const& key) const ←❷
        {
            return std::find_if(data.begin(),data.end(),
                                [&](bucket_value const& item)
                                {return item.first==key;});
        }
    public:
        Value value_for(Key const& key,Value const& default_value) const
        {
            std::shared_lock<std::shared_mutex> lock(mutex); ←❸
            bucket_iterator const found_entry=find_entry_for(key);
            return (found_entry==data.end())?
                default_value:found_entry->second;
        }
        void add_or_update_mapping(Key const& key,Value const& value)
        {
            std::unique_lock<std::shared_mutex> lock(mutex); ←❹
            bucket_iterator const found_entry=find_entry_for(key);
            if(found_entry==data.end())
            {
                data.push_back(bucket_value(key,value));
            }
            else
            {
                found_entry->second=value;
            }
        }
        void remove_mapping(Key const& key)
        {
            std::unique_lock<std::shared_mutex> lock(mutex); ←❺
            bucket_iterator const found_entry=find_entry_for(key);
            if(found_entry!=data.end())
            {
                data.erase(found_entry);
            }
        }
    };
};
```



```

std::vector<std::unique_ptr<bucket_type> > buckets;           ←❶
Hash hasher;
bucket_type& get_bucket(Key const& key) const                ←❷
{
    std::size_t const bucket_index=hasher(key)%buckets.size();
    return *buckets[bucket_index];
}
public:
typedef Key key_type;
typedef Value mapped_type;
typedef Hash hash_type;
threadsafe_lookup_table(
    unsigned num_buckets=19,Hash const& hasher_=Hash()):
    buckets(num_buckets),hasher(hasher_)
{
    for(unsigned i=0;i<num_buckets;++i)
    {
        buckets[i].reset(new bucket_type);
    }
}
threadsafe_lookup_table(threadsafe_lookup_table const& other)=delete;
threadsafe_lookup_table& operator=(
    threadsafe_lookup_table const& other)=delete;
Value value_for(Key const& key,
    Value const& default_value=Value()) const
{
    return get_bucket(key).value_for(key,default_value); ←❸
}
void add_or_update_mapping(Key const& key,Value const& value)
{
    get_bucket(key).add_or_update_mapping(key,value); ←❹
}
void remove_mapping(Key const& key)
{
    get_bucket(key).remove_mapping(key); ←❺
}
};

```

В этой реализации для хранения сегментов используется вектор `std::vector<std::unique_ptr<bucket_type>>` ❶, позволяющий задавать в конструкторе количество сегментов. По умолчанию берется произвольное простое число 19 — с количеством сегментов, выраженными простыми числами, хеш-таблицы работают лучше. Каждый сегмент защищен экземпляром мьютекса `std::shared_mutex` ❶, допускающим выполнение множества конкурентных считываний или единственный вызов любой из функций, вносящих изменения в отношении каждого сегмента.

Поскольку количество сегментов зафиксировано, функция `get_bucket()` ❷ может вызываться без блокировок (❸, ❹ и ❺), а затем мьютекс сегмента можно заблокировать либо для совместного владения (только для чтения) ❹, либо для исключительного владения (для чтения — записи) (❸ и ❹) в зависимости от применимости к каждой функции.

Чтобы определить наличие записи в сегменте, всеми тремя функциями в отношении сегмента используется компонентная функция `find_entry_for()` ❷. В каждом

сегменте содержится всего лишь список пар «ключ — значение» `std::list<>`, поэтому добавление и удаление записей выполняется легко.

Одновременность доступа и надлежащую защиту данных с помощью блокировок мьютексов мы рассмотрели, а как насчет безопасности выдачи исключений? Функция `value_for` не вносит никаких изменений, значит, с ней все в порядке: если она выдаст исключение, это не повлияет на структуру данных. Функция `remove_mapping` вносит изменения в список за счет вызова функции `erase`, но она гарантированно не выдает исключение, стало быть, в ней тоже все безопасно. Остается только функция `add_or_update_mapping`, которая может выдать исключение в любой из двух ветвей `if`. Функция `push_back` в этом смысле безопасна и в случае выдачи исключений сохранит исходное состояние списка в неприкосновенности. Следовательно, и с этой ветвью все в порядке. Единственную проблему создает присваивание в случае замены существующего значения: если оператор присваивания выдаст исключение, остается полагаться на то, что он не изменит исходное состояние. Но это не повлияет на структуру данных в целом, так как имеет отношение только к типу, предоставленному пользователем, на которого свободно можно возложить решение данной проблемы.

В начале раздела уже упоминалось, что одной из предпочтительных особенностей такой поисковой таблицы было бы получение мгновенного образа ее текущего состояния, например, в виде объекта `std::map<>`. Для этого может потребоваться заблокировать весь контейнер, чтобы обеспечить извлечение целостной копии состояния, и без блокировки всех сегментов здесь не обойтись. Поскольку обычные операции над поисковой таблицей требуют в конкретный момент блокировки только одного сегмента, это станет единственной операцией, запрашивающей блокировки сразу всех сегментов. Поэтому их блокировка в одном и том же порядке (например, с приращением индекса сегмента) делает невозможной взаимную блокировку. Соответствующая реализация показана в листинге 6.12.

Листинг 6.12. Получение содержимого `threadsafe_lookup_table` в виде объекта `std::map<>`

```
std::map<Key, Value> threadsafe_lookup_table::get_map() const
{
    std::vector<std::unique_lock<std::shared_mutex> > locks;
    for(unsigned i=0; i<buckets.size(); ++i)
    {
        locks.push_back(
            std::unique_lock<std::shared_mutex>(buckets[i].mutex));
    }
    std::map<Key, Value> res;
    for(unsigned i=0; i<buckets.size(); ++i)
    {
        for(bucket_iterator it=buckets[i].data.begin();
            it!=buckets[i].data.end();
            ++it)
        {
            res.insert(*it);
        }
    }
    return res;
}
```

Реализация поисковой таблицы, показанная в листинге 6.11, расширяет возможности одновременного доступа к данным всей таблицы за счет блокировки каждого сегмента по отдельности и использования мьютекса `std::shared_mutex`, позволяющего выполнять конкурентное чтение каждого сегмента. А нельзя ли расширить возможности одновременного доступа к данным с помощью более подробной детализации блокировки? Ответ на этот вопрос будет дан в следующем разделе при разборе решения на основе применения потокобезопасного контейнера списка с поддержкой итератора.

6.3.2. Создание потокобезопасного списка с использованием блокировок

Список является одной из наиболее востребованных структур данных, поэтому стремление создать потокобезопасный список не вызывает вопросов. Все зависит от того, какие средства вам нужны, и следует выбрать средство с поддержкой итераторов, от добавления которого в отображение (`map`) я отказался, так как оно было слишком сложным. Основной проблемой поддержки итератора в стиле STL является необходимость сохранения в нем какой-либо ссылки на внутреннюю структуру данных контейнера. Если контейнер можно изменить из другого потока, эта ссылка должна оставаться действительной, что потребует удержания итератором блокировки на какую-то часть структуры. Учитывая, что время жизни итератора STL-стиля никак не контролируется контейнером, от применения такого итератора придется отказаться.

Альтернативой станет включение функций итерации, таких как `for_each`, в сам контейнер в качестве его составной части. Тем самым вся ответственность за итерацию и блокировку будет возложена на контейнер, но это противоречит рекомендациям по предотвращению взаимных блокировок, изложенным в главе 3. Чтобы функция `for_each` выполняла полезную работу, она должна вызывать код, предоставленный пользователем, и при этом удерживать внутреннюю блокировку. Но этим дело не ограничивается. Она также должна передавать ссылку на каждый элемент контейнера пользовательскому коду, чтобы тот мог обратиться к этому элементу. Этого можно избежать, передав пользовательскому коду копии всех элементов, но если элементы данных будут большими, такой вариант станет слишком затратным.

Итак, на данный момент ответственность за предотвращение взаимных блокировок из-за установки блокировок в операциях, предоставленных пользователем, а также за недопущение состояний гонки за данными из-за хранения ссылок для обращений к ним вне блокировок будет возложена на пользователя. При использовании списка таблицей поиска гарантируется полная безопасность, поскольку вам известно, что ничего предосудительного происходить не будет.

Остается только вопрос о составе операций, предлагаемых списком. Если вернуться к коду листингов 6.11 и 6.12, можно понять, какого рода операции потребуются:

- ☐ добавление элемента к списку;
- ☐ удаление из списка элемента, отвечающего конкретному условию;

- ❑ поиск в списке элемента, отвечающего конкретному условию;
- ❑ обновление элемента, отвечающего конкретному условию;
- ❑ копирование всех элементов списка в другой контейнер.

Чтобы списочный контейнер приобрел универсальность, было бы неплохо добавить еще несколько операций, например вставку элемента в конкретное место, но для нашей поисковой таблицы этого не нужно, поэтому пусть решение данной задачи станет упражнением для читателей.

Замысел, положенный в основу более подробной детализации блокировки связанного списка, заключается в наличии мьютекса для каждого узла. Если список растет, мьютексов становится слишком много! Преимущество данного подхода состоит в том, что операции над отдельными частями списка выполняются действительно одновременно: каждая операция удерживает блокировки только тех узлов, которые ее интересуют, и снимает блокировку каждого узла по мере перемещения к следующему узлу. Реализация такого списка показана в листинге 6.13.

Листинг 6.13. Потокобезопасный список с поддержкой итерации

```
template<typename T>
class threadsafe_list
{
    struct node          ← ❶
    {
        std::mutex m;
        std::shared_ptr<T> data;
        std::unique_ptr<node> next;
        node():          ← ❷
            next()
        {}
        node(T const& value): ← ❸
            data(std::make_shared<T>(value))
        {}
    };
    node head;
public:
    threadsafe_list()
    {}
    ~threadsafe_list()
    {
        remove_if([](node const&){return true;});
    }
    threadsafe_list(threadsafe_list const& other)=delete;
    threadsafe_list& operator=(threadsafe_list const& other)=delete;
    void push_front(T const& value)
    {
        std::unique_ptr<node> new_node(new node(value)); ← ❹
        std::lock_guard<std::mutex> lk(head.m);          ← ❺
        new_node->next=std::move(head.next);             ← ❻
        head.next=std::move(new_node);
    }
};
template<typename Function>
```

```

void for_each(Function f) ← 7
{
    node* current=&head;
    std::unique_lock<std::mutex> lk(head.m); ← 8
    while(node* const next=current->next.get()) ← 9
    {
        std::unique_lock<std::mutex> next_lk(next->m); ← 10
        lk.unlock(); ← 11
        f(*next->data); ← 12
        current=next;
        lk=std::move(next_lk); ← 13
    }
}

template<typename Predicate>
std::shared_ptr<T> find_first_if(Predicate p) ← 14
{
    node* current=&head;
    std::unique_lock<std::mutex> lk(head.m);
    while(node* const next=current->next.get())
    {
        std::unique_lock<std::mutex> next_lk(next->m);
        lk.unlock();
        if(p(*next->data)) ← 15
        {
            return next->data; ← 16
        }
        current=next;
        lk=std::move(next_lk);
    }
    return std::shared_ptr<T>();
}

template<typename Predicate>
void remove_if(Predicate p) ← 17
{
    node* current=&head;
    std::unique_lock<std::mutex> lk(head.m);
    while(node* const next=current->next.get())
    {
        std::unique_lock<std::mutex> next_lk(next->m);
        if(p(*next->data)) ← 18
        {
            std::unique_ptr<node> old_next=std::move(current->next);
            current->next=std::move(next->next); ← 19
            next_lk.unlock();
        } ← 20
        else
        {
            lk.unlock(); ← 21
            current=next;
            lk=std::move(next_lk);
        }
    }
}
};

```


Шаблон `threadsafe_list<>` из листинга 6.13 является однонаправленным списком, где каждый элемент представлен структурой типа `node` (узел) ❶. В качестве головы списка `head`, изначально имеющей `next`-указатель со значением `NULL` ❷, используется сконструированная с установками по умолчанию структура `node`. Новые узлы добавляются с помощью функции `push_front()`: сначала создается новый узел ❸ и под сохраняемые в нем данные распределяется место в куче ❹, а указателю `next` присваивается значение `NULL`. Затем нужно заблокировать мьютекс для узла `head`, чтобы получить соответствующее значение указателя `next` ❺, и вставить узел в начало списка, установив для `head.next` значение указателя на новый узел ❻. Пока нас все устраивает: чтобы добавить к списку новую запись, требуется заблокировать всего один мьютекс, следовательно, нет риска взаимной блокировки. К тому же медленное распределение памяти происходит вне блокировки: получается, что блокировка защищает только обновление двух значений указателей, что не может дать сбой. Теперь перейдем к функциям, выполняющим итерацию.

Сначала рассмотрим `for_each()` ❷. Эта операция получает функцию некоторого типа, применяемую ко всем элементам списка. Как и в большинстве стандартных библиотечных алгоритмов, она получает эту функцию по значению и будет работать вместе либо с ней, либо с объектом типа, имеющим оператор вызова функции. В данном случае функция должна принимать в качестве единственного параметра значение типа `T`. Здесь выполняется эстафетная блокировка с перекрытием. Сначала блокируется мьютекс на узле `head` ❸. После этого можно будет безопасно получить указатель на следующий узел `next` (используя `get()`, поскольку владение указателем не принимается). Если значение этого указателя не `NULL` ❹, блокируется мьютекс на этом узле ❺, чтобы обработать данные. Получив блокировку на данном узле, можно снять блокировку с предыдущего узла ❻ и вызвать указанную функцию ❼. Когда функция завершит работу, можно будет обновить указатель `current` на обработанный узел и с помощью функции `move` передать владение блокировкой от `next_1k` в `1k` ❽. Поскольку `for_each` передает каждый элемент данных непосредственно предоставленной функции, можно при необходимости воспользоваться этим для обновления элемента или его копирования в другой контейнер либо сделать все, что угодно. Если отклонений в поведении функции не будет, никакой опасности не возникнет, поскольку мьютекс для узла, содержащего элемент данных, удерживается в течение всего вызова.

Функция `find_first_if()` ❹ похожа на `for_each()`, основное отличие в том, что предоставляемый предикат должен возвращать `true`, чтобы показать соответствие, или `false`, если оно не найдено ❺. Как только соответствующий элемент будет определен, возвращаются содержащиеся в нем данные ❻, а поиск прекращается. То же самое можно было бы сделать с помощью функции `for_each()`, но после найденного соответствия она продолжит ненужную обработку остальной части списка.

Функция `remove_if()` ❼ немного отличается от предыдущей функции, поскольку она должна обновлять список. Воспользоваться для этого функцией `for_each()` нельзя. Если предикат возвращает `true` ❶, узел из списка удаляется путем обновления значения `current->next` ❷. После удаления блокировку мьютекса для узла `next` можно снять. Узел удаляется, когда объект `std::unique_ptr<node>`, в который

он был перемещен, выходит из области видимости 20. В данном случае значение `current` не обновляется, так как необходимо проверить новый узел `next`. Если предикат возвращает `false`, нужно переключаться по списку, как и раньше 21.

Возможно ли при таком количестве мьютексов возникновение взаимных блокировок или состояний гонки? При правильном поведении предоставляемых предикатов и функций это полностью исключается. Итерация всегда проходит в одном направлении, всегда начинается с узла `head` и всегда блокирует следующий мьютекс, прежде чем разблокировать текущий, следовательно, иной порядок блокировки в других потоках полностью исключен. Единственным возможным кандидатом на провоцирование состояния гонки является удаление намеченного для этого узла в функции `remove_if()` 22, поскольку оно выполняется после разблокировки мьютекса (уничтожение заблокированного мьютекса приводит к неопределенному поведению). Но если немного подумать, можно прийти к выводу, что это абсолютно безопасно, так как на прежнем узле (`current`) мьютекс все еще удерживается, поэтому никакой новый поток не сможет предпринять попытку завладеть блокировкой на удаляемом узле.

А как насчет возможностей конкурентности? Вся эта более подробная детализация затевалась для расширения именно этих возможностей по сравнению с тем, которые появлялись при использовании одного мьютекса. Удалось ли достичь желаемого? Конечно, удалось: разные потоки могут одновременно работать с разными узлами списка, обрабатывая каждый элемент с помощью `for_each()`, выполняя поиск с применением `find_first_if()` или удаляя элементы с помощью `remove_if()`. Но поскольку мьютексы для каждого узла должны блокироваться по порядку, потоки не могут обгонять друг друга. Если один поток тратит много времени на обработку конкретного узла, другим потокам, подошедшим именно к этому узлу, придется ждать.

Резюме

В начале главы речь шла о том, что подразумевается под разработкой структуры данных для реализации конкурентности, и приводились соответствующие рекомендации. Затем были рассмотрены примеры практического применения этих рекомендаций при реализации наиболее востребованных структур данных с возможностью конкурентных обращений (стек, очередь, хеш-таблица и связанный список) за счет использования блокировок и предотвращения состояний гонки за доступ к данным. Теперь вы сможете оценить конструкцию собственной структуры данных и подумать, где в ней имеется потенциал для конкурентности и где может возникнуть состояние гонки.

В главе 7 будет показано, как при соблюдении прежнего набора рекомендаций отказаться от блокировок и реализовать необходимые ограничения, направленные на упорядочение доступа к данным за счет низкоуровневых атомарных операций.